

Recurrent Neural Network – GRU - LSTM

Recurrent Neural Network

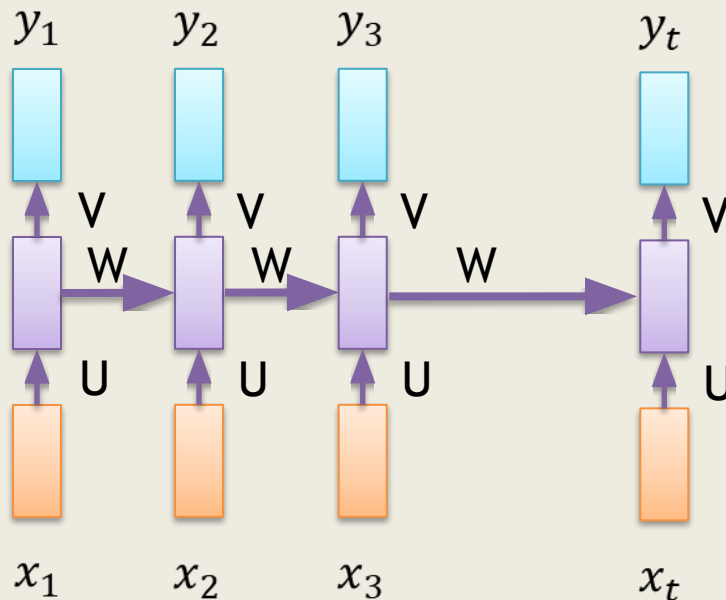
- **A recurrent neural network (RNN) is a type of artificial neural network.**
- **Designed to recognize a data's sequential characteristics and use patterns to predict the next likely scenario.**

Recurrent Neural Network

- Account for dependence between inputs
- Account for variable number of inputs
- Make sure that the function executed at each time step is the same

Recurrent Neural Network

- Account for dependence



$$s_t = \tanh(Ux_t + Ws_{t-1} + b)$$

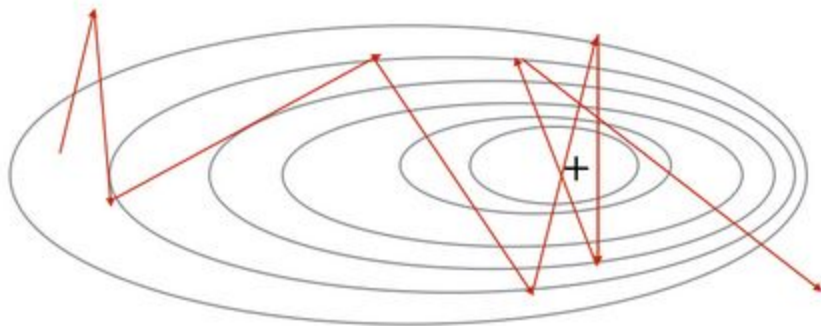
$$y_i = \mathcal{O}(Vs_t + c)$$

Where s_t is the state of the Network at timestep t
Parameters W, U, V, c, b are shared across timesteps

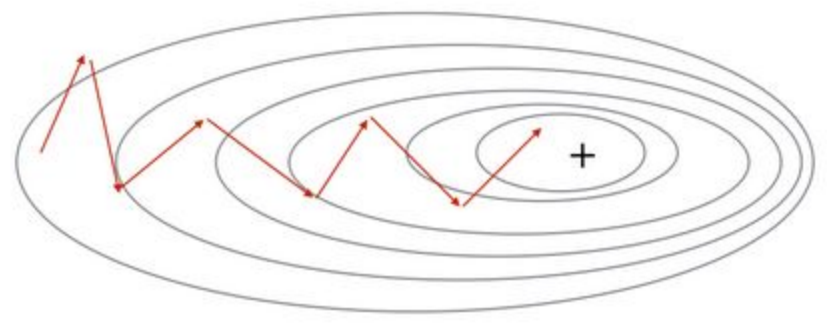
Challenges

- Vanishing Gradient - Alternate Architectures - GRU (2014), LSTM (1997)
- Exploding Gradient - Gradient Clipping
- Backpropagation is Time consuming - TBPTT (A similar Mini Batch Gradient Descent process)

Without gradient clipping



With gradient clipping

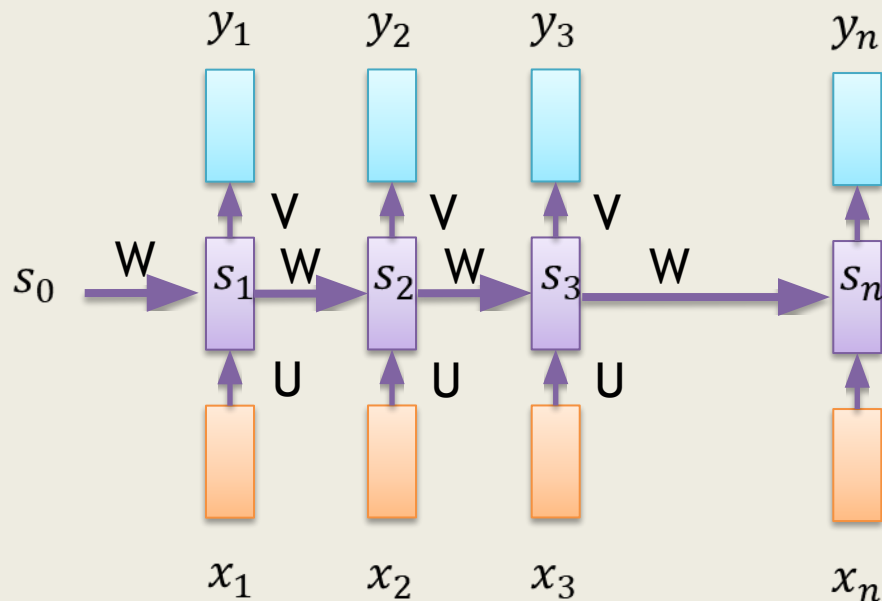


Challenges

- Suppose we want to predict the blank word in the text
- ‘David, a 36-year old man lives in San Francisco. He has a female friend Maria. Maria works as a cook in a famous restaurant in New York whom he met recently in a school alumni meet. Maria told him that she always had a passion for _____ .
- What may be the prediction here
- Here, the network to learn from dependency ‘cook’ to predict ‘cooking.
- There is a gap between the information what we want to predict and from where we want it to get predicted . This is called long-term dependency.

Gated Recurrent Unit (GRU)

- Need some memory - Short term and long term memory
- RNN stores only a short term memory.
- For example, S_3 depends on S_2 and not explicitly on S_1



GRU

- LSTM - Separate cell for memory
- GRU - Some memory like operations

Simplified GRU (Prof. Andrew Ng)

- Not a practiced algorithm

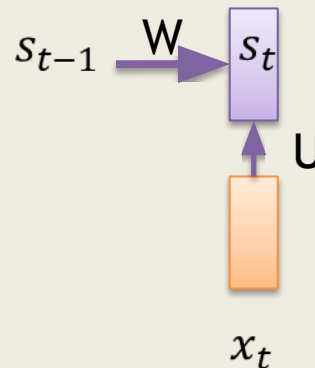
$$S_t = \tanh(Ux_t + WS_{t-1})$$

↓ ↓
Output of the RNN state

$$g = \tanh(Ux_t + WS_{t-1}) \quad [\text{Simply Activation Function}]$$

$$g = \tanh(z_g) \text{ where } z_g = Ux_t + WS_{t-1}$$

$$g = \tanh(z_g) \text{ where } z_g = U_g x_t + W_g S_{t-1}$$



Simplified GRU (Prof. Andrew Ng)

- Simplified GRU - linear combination of RNN output and older computations

$$S_t = (1 - \lambda)g + \lambda S_{t-1} \quad \lambda \in [0,1]$$

If $\lambda = 1$ then $S_t = S_{t-1}$ A pure memory

If $\lambda = 0$ then $S_t = g$ A Output of RNN state

So a Simplified GRU will be $S_t = (1 - f) \odot g + f \odot S_{t-1}$


Forget gate

What value of f to choose? $f \in [0,1]$

$$f = \sigma(Ux_t + WS_{t-1})$$

$$f = \sigma(Z_f) \text{ where } (U_f x_t + W_f S_{t-1})$$

A Simplified GRU

Let us combine together to form

$$S_t = (1 - f) \odot g + f \odot S_{t-1}$$

$$g = \tanh(z_g) \text{ where } z_g = U_g x_t + W_g S_{t-1}$$

$$f = \sigma(Z_f) \text{ where } (U_f x_t + W_f S_{t-1})$$

Simplified GRU (Prof. Andrew Ng)

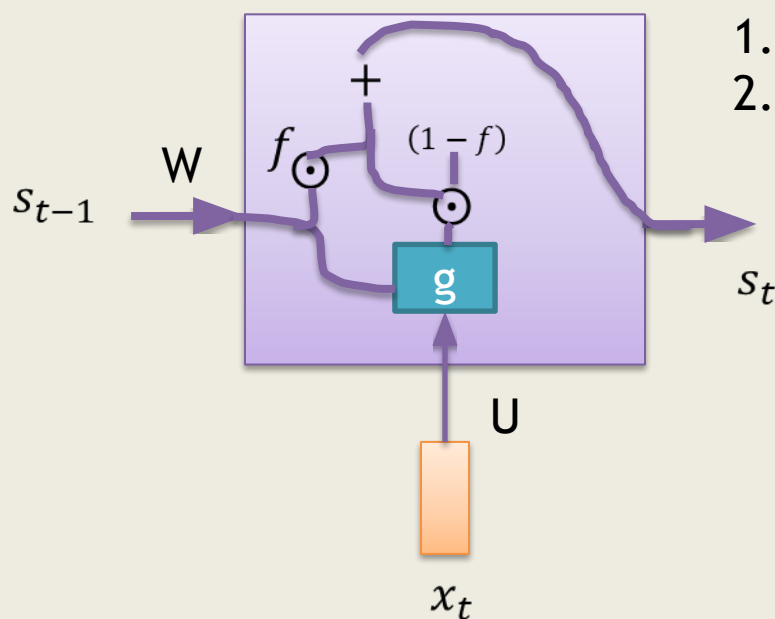
A Simplified GRU

Let us combine together to form

$$S_t = (1 - f) \odot g + f \odot S_{t-1}$$

$$g = \tanh(z_g) \text{ where } z_g = U_g x_t + W_g S_{t-1}$$

$$f = \sigma(z_f) \text{ where } (U_f x_t + W_f S_{t-1})$$



Forget Gate is a switch

1. To take older computations
2. To take current output

Gated Recurrent Unit (GRU)

$$S_t = (1 - f) \odot g' + f \odot S_{t-1}$$

$$f = \sigma(Z_f) \text{ where } (U_f x_t + W_f S_{t-1})$$

$$g' = \tanh(z_g)$$

$$\text{Where } z_g = U_g x_t + W_g (r \odot S_{t-1})$$

Works better for Deeper
networks



Remember Gate

$$r = \sigma(Z_r) \text{ where } (U_r x_t + W_r S_{t-1})$$

Reset/ Remember Gate(r): It determines how much of the past knowledge to remember.

Forget/ Update Gate(f): It determines how much of the past knowledge needs to be passed along into the future.

Long Short Term Memory (LSTM) Model

- LSTM - Long Short Term Memory networks - introduced by Hochreiter and Schmiduber [R1]
- Widely been used for
 - speech recognition
 - language modeling
 - sentiment analysis
 - Text prediction
 - Time series classification
 - Time Series Prediction

R1 : Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.

Long Short-Term Memory (LSTM)

Requires a Memory cell (C_t)

$$C_t = f \odot C_{t-1} + i \odot g \quad \text{where } f \in [0,1] \quad i \in [0,1]$$
$$S_t = O \odot \tanh(C_t) \quad \text{where } O \in [0,1]$$

f – forget gate
 i – Input gate
 O – Output gate

Finally

$$S_t = O \odot \tanh(C_t)$$
$$C_t = f \odot C_{t-1} + i \odot g$$
$$O = \sigma(Z_o) \quad Z_o = (U_o x_t + W_o S_{t-1})$$
$$f = \sigma(Z_f) \quad Z_f = (U_f x_t + W_f S_{t-1})$$
$$i = \sigma(Z_i) \quad Z_i = (U_i x_t + W_i S_{t-1})$$
$$g = \tanh(Z_g) \quad Z_g = (U_g x_t + W_g S_{t-1})$$

RNN Requires
2 Weight Matrixes
Simplified GRU
4 Weight Matrixes
Full GRU
6 weight Matrixes
LSTM
8 Weight Matrixes

LSTM > GRU > RNN in training the number of layers a network

LSTM

- **Forget gate** - Decides what information should be thrown away or kept. $Z_f \in [0,1]$.
- The closer to 0 means to forget, and the closer to 1 means to keep.

$$f = \sigma(Z_f) \quad Z_f = (U_f x_t + W_f S_{t-1})$$

- **Input Gate** - Used to update the cell state. $Z_i \in [0,1]$.
- Values will be updated by transforming the values to be between 0 and 1. 0 means not important, and 1 means important.

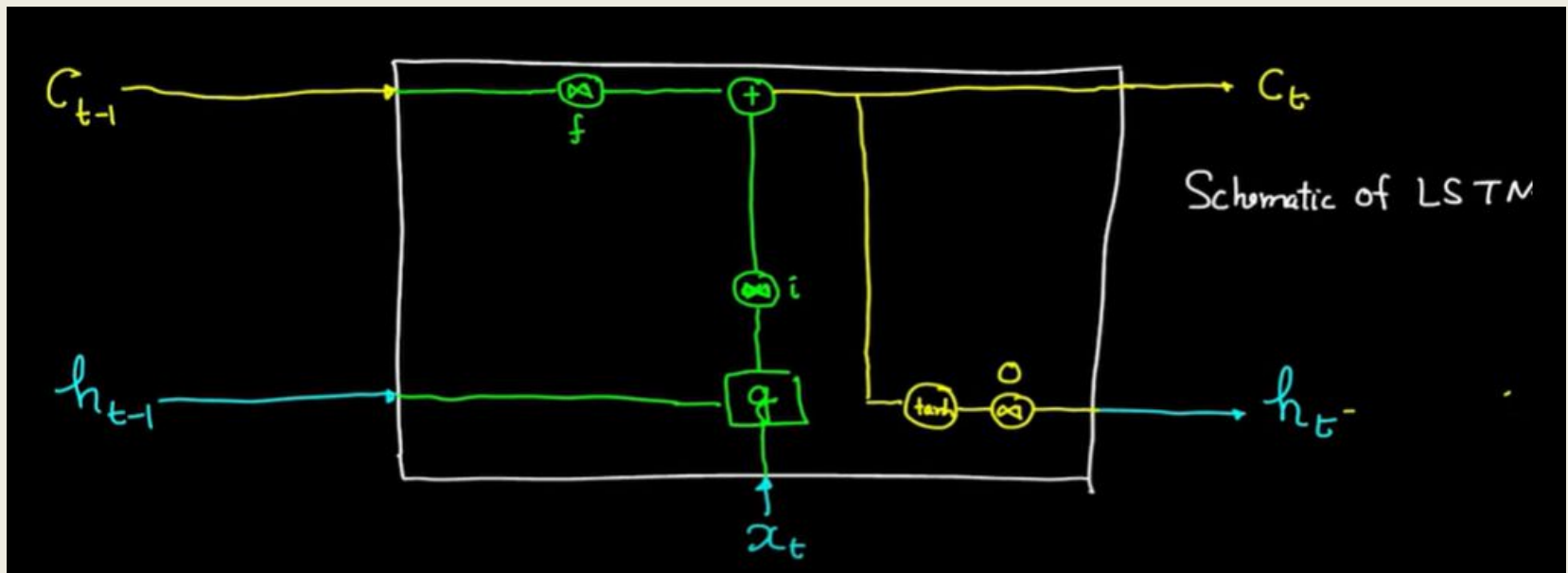
$$i = \sigma(Z_i) \quad Z_i = (U_i x_t + W_i S_{t-1})$$

LSTM

- **Output Gate**
- The output gate decides what the next hidden state should be. Contains information on previous inputs. The hidden state is also used for predictions.

$$O = \sigma(Z_o) \quad Z_o = (U_o x_t + W_o S_{t-1})$$

LSTM



LSTM – Forward Pass

$$W = \begin{bmatrix} U_i & W_i \\ U_g & W_g \\ U_f & W_f \\ U_o & W_o \\ W_y & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.25 \\ 0.3 & 0.4 \\ 0.03 & 0.06 \\ 0.02 & 0.04 \\ 0.6 & 0 \end{bmatrix} \quad \text{Assume } x_1 = 0.1 \text{ and } x_2 = 0.2, y_k = 0.08$$

Initially, $s_o = 0, c_o = 0$

Finally

$$\begin{aligned} S_t &= O \odot \tanh(C_t) \\ C_t &= f \odot C_{t-1} + i \odot g \\ O &= \sigma(Z_o) \quad Z_o = (U_o x_t + W_o S_{t-1}) \\ f &= \sigma(Z_f) \quad Z_f = (U_f x_t + W_f S_{t-1}) \\ i &= \sigma(Z_i) \quad Z_i = (U_i x_t + W_i S_{t-1}) \\ g &= \tanh(Z_g) \quad Z_g = (U_g x_t + W_g S_{t-1}) \end{aligned}$$

LSTM – Forward Pass

$$W = \begin{bmatrix} U_i & W_i \\ U_g & W_g \\ U_f & W_f \\ U_o & W_o \\ W_y & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.25 \\ 0.3 & 0.4 \\ 0.03 & 0.06 \\ 0.02 & 0.04 \\ 0.6 & 0 \end{bmatrix} \quad \text{Assume } x_1 = 0.1 \text{ and } x_2 = 0.2, y_k = 0.08$$

Time stamp t=1

$$i_1 = \sigma(Z_i) \quad Z_i = (U_i x_t + W_i S_{t-1}) \Rightarrow \sigma(0.5 * 0.1 + 0.25 * 0) = 0.512$$

$$f_1 = \sigma(Z_f) \quad Z_f = (U_f x_t + W_f S_{t-1}) \Rightarrow \sigma(0.03 * 0.1 + 0.06 * 0) = 0.500$$

$$O_1 = \sigma(Z_o) \quad Z_o = (U_o x_t + W_o S_{t-1}) \Rightarrow \sigma(0.02 * 0.1 + 0.04 * 0) = 0.5004$$

$$g_1 = \tanh(Z_g) \quad Z_g = (U_g x_t + W_g S_{t-1}) \Rightarrow \sigma(0.3 * 0.1 + 0.4 * 0) = 0.0299$$

$$C_t = f \odot C_{t-1} + i \odot g \quad ; C_1 = f_1 \odot C_0 + i_1 \odot g_1 = (0.5 * 0 + 0.512 * 0.0299)$$

$$C_1 = 0.0153$$

$$S_t = O \odot \tanh(C_t) \quad S_1 = O_1 \odot \tanh(C_1) = 0.5004 \odot \tanh(0.0153)$$

$$S_1 = 0.0076$$

LSTM – Forward Pass

$$W = \begin{bmatrix} U_i & W_i \\ U_g & W_g \\ U_f & W_f \\ U_o & W_o \\ W_y & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.25 \\ 0.3 & 0.4 \\ 0.03 & 0.06 \\ 0.02 & 0.04 \\ 0.6 & 0 \end{bmatrix} \quad \text{Assume } x_1 = 0.1 \text{ and } x_2 = 0.2, y_k = 0.08$$

Time stamp t=2

$$i_2 = \sigma(Z_i) \quad Z_i = (U_i x_t + W_i S_{t-1}) \Rightarrow \sigma(0.5 * 0.2 + 0.25 * 0.0076) = 0.525$$

$$f_2 = \sigma(Z_f) \quad Z_f = (U_f x_t + W_f S_{t-1}) \Rightarrow \sigma(0.03 * 0.1 + 0.06 * 0.0076) = 0.501$$

$$O_2 = \sigma(Z_o) \quad Z_o = (U_o x_t + W_o S_{t-1}) \Rightarrow \sigma(0.02 * 0.1 + 0.04 * 0.0076) = 0.554$$

$$g_2 = \tanh(Z_g) \quad Z_g = (U_g x_t + W_g S_{t-1}) \Rightarrow \sigma(0.3 * 0.1 + 0.4 * 0.0076) = 0.0629$$

$$C_t = f \odot C_{t-1} + i \odot g \quad ; \quad C_2 = f_2 \odot C_1 + i_2 \odot g_2 = (0.501 * 0.0153 + 0.525 * 0.0629)$$

$$C_1 = 0.0406$$

$$S_t = O \odot \tanh(C_t) \quad S_2 = O_2 \odot \tanh(C_2) = 0.554 \odot \tanh(0.0406)$$

$$S_1 = 0.0224$$

Backward components

Δ_t the output difference as computed by an subsequent layers

Δs_{t+1} the computed by the next time-step LSTM

To Find

$$\delta s_t = \Delta_t + \Delta s_{t+1}$$

$$\delta c_t = \delta s_t \cdot o_t (1 - \tanh^2(c_t)) + \delta c_{t+1} \cdot f_{t+1}$$

$$\delta g_t = \delta c_t \cdot i_t \cdot (1 - g_t^2)$$

$$\delta i_t = \delta c_t \cdot g_t \cdot i_t \cdot (1 - i_t)$$

$$\delta f_t = \delta c_t \cdot c_{t-1} \cdot f_t \cdot (1 - f_t)$$

$$\delta o_t = \delta s_t \cdot \tanh(c_t) \cdot o_t \cdot (1 - o_t)$$

$$\delta x_t = U^T \cdot \delta gates_t$$

$$\Delta s_{t-1} = W^T \cdot \delta gates_t$$

$$\delta U = \sum_{t=0}^T \delta gates_t \cdot x_t$$

$$\delta W = \sum_{t=0}^T \delta gates_t \cdot s_t$$

Backward @ t=2

First let us compute the difference in output from the expected label

$$\text{Loss (L2)} \quad E(y, \hat{y}) = \frac{(y - \hat{y})^2}{2}$$

$$\text{The derivative w.r.t } x \text{ is} \quad \Delta_t = \delta y E = (y - \hat{y}) = 0.01344 - 0.08 = -0.06656$$

as there are no future time steps in Backward propagation $\Delta c_t = 0$

Backward @ t=2

$$\delta c_t = \Delta_t + \Delta c_t = -0.06656 - 0 = -0.06656$$

$$\delta s_t = \delta c_t \cdot o_t (1 - \tanh^2(s_t)) + \delta s_{t+1} \cdot f_{t+1} = -0.06656 \cdot 0.554 \cdot (1 - \tanh^2(0.0224)) + 0.0$$

$$\delta g_t = \delta s_t \cdot i_t \cdot (1 - g_t^2) = 0.0224 \cdot 0.525 \cdot (1 - 0.0629^2) =$$

$$\delta i_t = \delta s_t \cdot g_t \cdot (1 - i_t) = 0.0224 \cdot 0.0629 \cdot (1 - 0.525) =$$

$$\delta f_t = \delta s_t \cdot s_{t-1} \cdot f_t (1 - f_t) = 0.0224 \cdot 0.0076 \cdot 0.501 \cdot (1 - 0.501) =$$

$$\delta o_t = \delta c_t \cdot \tanh(s_t) \cdot o_t (1 - o_t) = 0.06656 \cdot \tanh(0.0224) \cdot 0.554 \cdot (1 - 0.554)$$

$$\delta x_t = U^T \cdot \delta gates_t$$

$$\Delta out_{t-1} = U^T \cdot \delta gates_t$$

THANK YOU